School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

BCSE307P Compiler Design Lab Project

L11+L12

INTERACTIVE GUI FOR OPERATOR PRECEDENCE PARSING AND CONVERSION OF RE TO DFA USING DIRECT METHOD

23BCE0661 **SAMRIDDHI TIWARY**

PROJECT REVIEW 2

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

ABSTRACT

This project develops an interactive Compiler Design visualization tool for Operator Precedence Parsing and direct conversion of Regular Expressions to Deterministic Finite Automata. Review 2 extends the Review-1 implementation from grammar analysis and precedence-table construction to a generic operator-precedence string parser. The system accepts a user-defined grammar, validates operator-grammar restrictions, computes LEADING and TRAILING sets, constructs a conflict-aware precedence table, tokenizes the input string, and performs step-by-step SHIFT/REDUCE parsing. To avoid hard-coded dependence on the common E/T/F arithmetic grammar, grammar productions are represented using typed tokens and the design supports both compact symbols and multi-character names such as Expr, Term, Factor, id, and num. During parsing, the topmost terminal on the stack is compared with the next input terminal. The relations '<' and '=' cause SHIFT, while '>' causes REDUCE after locating the nearest '<' handle boundary. The candidate handle is matched against grammar productions, with unit-production reachability and parser-cycle protection used to avoid incorrect or non-progressing reductions. Every step records the stack, remaining input, precedence relation, action, handle, production, and explanation. Invalid grammars, table conflicts, unknown terminals, missing relations, unmatched handles, ambiguous reductions, and repeated parser states are reported explicitly. The implementation is generic for valid operator-precedence grammars expressible in the supported notation; arbitrary context-free grammars are outside the theoretical scope of operator-precedence parsing. Review 3 will integrate the direct RE-to-DFA module.

TABLE OF CONTENTS

Chapter No.	Contents	Page No.
	Abstract	i
1.	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 MOTIVATION	2
	1.3 SCOPE OF THE PROJECT	2
2.	PROJECT DESCRIPTION AND GOALS	3
	2.1 LITERATURE REVIEW	3
	2.2 GAPS IDENTIFIED	4
	2.3 OBJECTIVES	4
	2.4 PROBLEM STATEMENT	5
	2.5 PROJECT PLAN	5
3.	TECHNICAL SPECIFICATION	6
	3.1 REQUIREMENTS	6
	3.1.1 Functional	6
	3.1.2 Non-Functional	7
	3.2 SYSTEM SPECIFICATION	7
	3.2.1 Hardware Specification	7
	3.2.2 Software Specification	8
4.	DESIGN APPROACH AND DETAILS	9
	4.1 DATASETS	9
	4.2 SYSTEM ARCHITECTURE	9
	4.3 ALGORITHMS	10
	4.4 DESIGN	14
	4.4.1 Data Flow Diagram	14
	4.4.2 Class Diagram	15
5.	IMPLEMENTATION	16
	5.1 Module Description	16
	5.2 Testing	19
6.	RESULTS AND DISCUSSION	20
	6.1 Detailed Results	20
	6.2 Output Screenshots	22
	6.3 Performance Analysis and Evaluation	23
	REFERENCES	24

1.1 BACKGROUND

Bottom-up parsing constructs a parse from the input toward the start symbol by repeatedly recognizing and reducing handles. Operator Precedence Parsing is a specialized bottom-up method that uses precedence relations between terminal symbols to decide whether the parser should shift the next input token or reduce a handle already present on the stack. The method is applicable to operator-precedence grammars, a restricted class of context-free grammars that contains no epsilon productions, no adjacent non-terminals on the right-hand side, and no conflicting precedence relations.

In Review 1, the project implemented grammar entry, structural validation, LEADING and TRAILING set computation, step-by-step relation generation, precedence-table construction, and conflict detection. Review 2 uses that foundation to implement the actual string parser. The parser must be grammar-independent: all tokenization, relation lookup, handle discovery, and reduction decisions are derived from the grammar entered by the user rather than from a hard-coded expression grammar.

1.2 MOTIVATION

Manual operator-precedence parsing is difficult to trace because a correct reduction depends on the topmost terminal, the next input terminal, the relation stored in the table, and the correct left boundary of the handle. A black-box implementation that displays only ACCEPT or REJECT does not provide sufficient educational value. The project therefore visualizes every parser state and explains why each action occurs. The Review-2 upgrade also removes the one-character-symbol assumption so that grammars using names such as Expr, Term, Factor, id, and num can be represented correctly.

1.3 SCOPE OF THE PROJECT

- Review 2 retains all Review-1 modules: grammar parsing, operator-grammar validation, LEADING/TRAILING computation, precedence-table construction, and conflict detection.
- Review 2 adds generic grammar tokenization, input tokenization, topmost-terminal detection, correct handle-boundary discovery, grammar-based reduction,

unit-production handling, parser trace generation, and detailed ACCEPT/REJECT output.

- The parser is intended for valid operator-precedence grammars expressible in the supported notation. It is not a general parser for arbitrary context-free grammars.
- The Regular Expression to DFA direct-method branch remains outside Review 2 and is planned for Review 3.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

2.1.1 Operator Precedence Parsing Based

Aho, Lam, Sethi, and Ullman describe bottom-up parsing, handle recognition, and precedence-based parsing in which precedence relations between terminals guide parser actions. The theoretical model establishes the distinction between shifting and reducing and provides the basis for locating a handle through precedence boundaries.

2.1.2 Compiler Implementation Based

Cooper and Torczon emphasize modular compiler front ends and separation between lexical processing, grammar representation, and parsing. Appel similarly promotes implementation structures in which algorithmic state is kept separate from presentation. These ideas motivate the Review-2 architecture: grammar and input tokenization are separate from precedence construction, while the parser engine returns structured steps that the GUI renders independently.

2.2 GAPS IDENTIFIED

- Many classroom implementations support only one-character terminals and non-terminals.
- Sample programs often hard-code reductions for E, T, and F instead of deriving them from user productions.
- Final ACCEPT/REJECT output alone does not expose the precedence relation, handle boundary, or production used at each reduction.
- Unit productions can create repeated non-progressing reductions if parser state is not tracked.
- Error reporting is often incomplete for unknown terminals, missing relations, conflicting precedence cells, unmatched handles, and ambiguous reductions.

2.3 OBJECTIVES

- Implement a grammar-independent operator-precedence parser driven by the entered grammar and generated precedence table.

- Support compact educational grammars and multi-character token notation.
- Tokenize input using the terminal vocabulary of the grammar and longest-match rules.
- Perform correct SHIFT/REDUCE decisions using the topmost stack terminal.
- Locate handles using the nearest left-yields-precedence boundary.
- Match handles to productions without hard-coded expression rules.
- Display a complete step-by-step parsing trace and exact rejection reasons.
- Protect the application against infinite or non-progressing reduction cycles.

2.4 PROBLEM STATEMENT

To extend the existing interactive Operator Precedence Parsing tool into a complete and explainable string parser that can process any valid operator-precedence grammar supported by the tokenizer. The parser must derive each action from the generated precedence table, identify and reduce valid handles according to the grammar, visualize the complete trace, and terminate safely with an accurate ACCEPT or REJECT verdict.

2.5 PROJECT PLAN

Table 2.1 Project Plan

Review	Major Work	Status
Review 1	Grammar validation, LEADING/TRAILING, precedence table, conflict detection	Completed
Review 2	Generic tokenization, SHIFT/REDUCE parser, handle detection, trace, error handling	Current Review
Review 3	RE syntax tree, nullable/firstpos/lastpos/followpos, direct DFA construction, final integration	Planned

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional

- Accept grammar productions using compact notation and token-separated notation.
- Identify terminals and non-terminals without assuming that every symbol has length one.
- Reject epsilon productions, adjacent non-terminals, malformed productions, and undefined non-terminals.
- Compute LEADING and TRAILING sets and construct a conflict-aware precedence table.
- Tokenize the user input string using the grammar terminal vocabulary.
- Find the rightmost terminal on the parser stack before every precedence comparison.
- SHIFT when the relation is < or =; REDUCE when the relation is >.
- Find the handle by moving left through stack terminals until the nearest < boundary is found.
- Match the handle against grammar productions and apply a valid reduction.
- Display stack, input, relation, action, handle, production, and explanation for each step.
- Accept only when the entire input is consumed and the stack represents the start symbol, considering valid unit-production closure.
- Reject safely on conflicts, unknown input symbols, missing relations, unmatched handles, genuine ambiguous reductions, or parser cycles.

3.1.2 Non-Functional

- Correctness: parser behavior must be derived from the grammar and not from hard-coded sample traces.
- Explainability: every parser action must include the formal reason and the production used.

- Modularity: tokenizer, precedence engine, parser engine, and UI renderer must remain separated.
- Termination: repeated parser states and excessive step counts must be detected.
- Usability: the system must support Previous, Next Step, Run All, and Reset Parsing controls.
- Responsiveness: all algorithms execute locally in the browser without a backend.

3.2 SYSTEM SPECIFICATION

3.2.1 Hardware Specification

- Laptop or desktop computer with a modern web browser.
- Minimum 4 GB RAM; 8 GB recommended for development.
- Standard keyboard and display for interactive grammar and input entry.

3.2.2 Software Specification

Table 3.1 Software Specification

Software / Tool	Usage
React	Component-based user interface
Vite	Development and build environment
JavaScript	Grammar algorithms and parser implementation
CSS	User interface styling
Vitest	Algorithm and parser unit testing
Web Browser	Client-side execution and visualization

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 DATASETS

Not applicable for Project 8. The project is an algorithmic Compiler Design application and does not require a machine-learning or statistical dataset. The inputs are formal grammars and test strings supplied by the user. Deterministic grammar/input combinations are used as test cases.

4.2 SYSTEM ARCHITECTURE

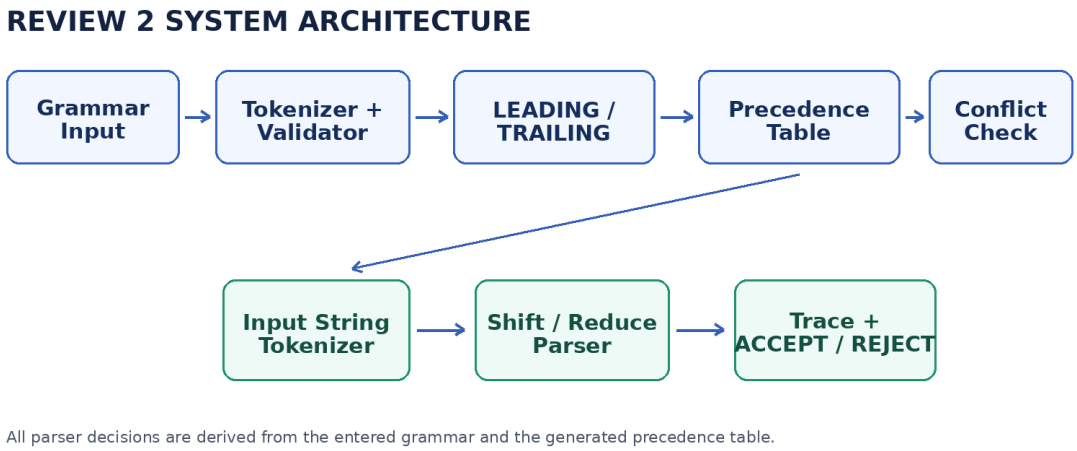


Figure 4.1 Review-2 System Architecture

The system is organized as a staged pipeline. The grammar tokenizer produces typed symbols, the validator enforces operator-grammar restrictions, LEADING/TRAILING and precedence-table modules provide the relations needed for parsing, and conflict detection ensures that the parser is enabled only for a deterministic table. The Review-2 input tokenizer and parser engine then generate a structured execution trace and final verdict.

4.3 ALGORITHMS

Algorithm 1: Grammar and Input Tokenization

	Input: Grammar text G, input string w
	Output: Typed grammar tokens and input token sequence
1	Read all production left-hand sides and build the known non-terminal vocabulary.

2	Tokenize each RHS using quoted terminals, whitespace-separated symbols, and longest matching known non-terminal names.
3	Store every RHS as an ordered array of typed TERMINAL and NON_TERMINAL objects.
4	Build the terminal vocabulary T from the parsed grammar.
5	For input w, ignore permitted whitespace and repeatedly choose the longest terminal in T matching the current input position.
6	If no terminal matches, return the unknown token and its position; otherwise continue until the entire input is consumed.

Algorithm 2: Topmost Terminal Detection

	Input: Parser stack K
	Output: Rightmost terminal a in K
1	Scan the parser stack from right to left.
2	Ignore NON_TERMINAL entries.
3	Return the first TERMINAL entry encountered.
4	If no terminal exists, report an invalid parser state.

Algorithm 3: Operator Precedence Shift/Reduce Parsing

	Input: Validated grammar G, input tokens w, precedence table M, start symbol S
	Output: Structured parsing trace and final ACCEPT/REJECT verdict
1	Initialize stack K = [$] and append $ to the input token list.
2	Let a be the topmost terminal in K and b be the next input terminal.
3	Look up relation M[a,b].
4	If M[a,b] is < or =, SHIFT b onto K, except when $=$ corresponds to the final acceptance boundary.
5	If M[a,b] is >, locate the handle boundary, extract the candidate handle, match it to grammar productions, and REDUCE it to the selected LHS.
6	If no relation, no production match, or an unresolved reduction ambiguity exists, terminate with REJECT and an explicit reason.
7	After every SHIFT or REDUCE, append a structured trace record containing stack, input, relation, action, handle, production, and explanation.
8	Accept only when the input is [$] and the stack contains the start symbol (or a unit-production-equivalent non-terminal) above $.

9	Abort safely if a parser state repeats without input progress or the maximum-step limit is exceeded.

Algorithm 4: Handle Boundary Detection

	Input: Current parser stack K and precedence table M
	Output: Start index of the candidate handle
1	List the positions of terminals present in K.
2	Start from the rightmost terminal position.
3	Move left from terminal to terminal and inspect the precedence relation between consecutive stack terminals.
4	Continue across = and internal handle structure.
5	Stop when the relation from the terminal immediately before the candidate handle to the first terminal inside the candidate is <.
6	Return every stack element after the boundary terminal, including intervening non-terminals, as the candidate handle.

Algorithm 5: Production Matching and Reduction

	Input: Candidate handle H and production set P
	Output: Selected production A -> beta or rejection reason
1	Compare H against every production RHS using exact terminal equality.
2	Prefer candidates whose non-terminal symbols match the stack symbols exactly.
3	When required by operator-form matching, treat non-terminal positions as placeholders while preserving terminals exactly.
4	Use unit-production closure to confirm whether a stack non-terminal is compatible with the expected grammar non-terminal.
5	If exactly one deterministic candidate remains, remove H and push its LHS.
6	If no candidate remains, reject with No Matching Handle.
7	If multiple valid candidates remain and cannot be deterministically resolved, reject with Ambiguous Reduction rather than choosing arbitrarily.

Algorithm 6: Unit Production Closure and Cycle Guard

	Input: Grammar G and current parser state
	Output: Reachability relation and safe termination condition

1	Collect all unit productions A -> B where both symbols are non-terminals.
2	Compute transitive reachability so that the parser knows which non-terminal can represent another through unit reductions.
3	Create a normalized parser-state key using the current stack and input index.
4	Record each visited state.
5	If the same state repeats without consuming input, terminate as a non-progressing parser cycle.
6	Also enforce a maximum parser-step limit proportional to input length and production count.

4.4 DESIGN

4.4.1 Data Flow Diagram

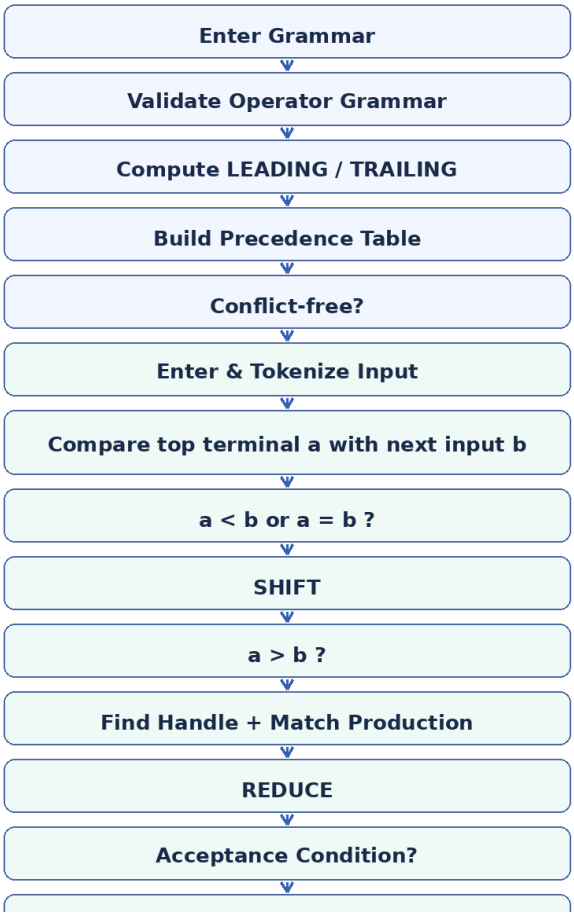


Figure 4.2 Data Flow / Parsing Workflow

4.4.2 Class Diagram

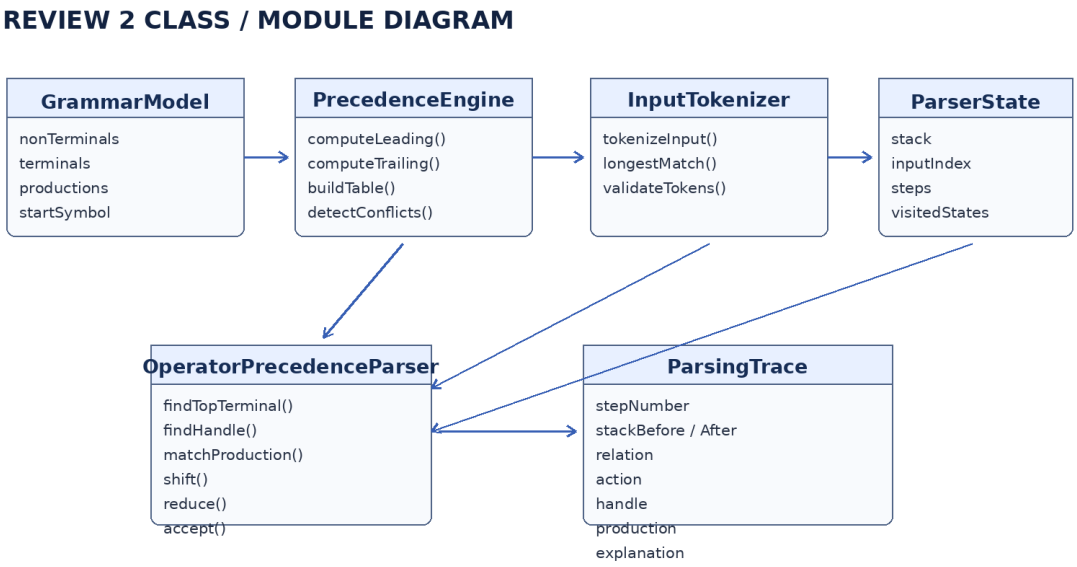


Figure 4.3 Class / Module Diagram

CHAPTER 5
IMPLEMENTATION

5.1 MODULE DESCRIPTION

5.1.1 Review-1 Foundation

The existing application already implements four functional stages: Grammar Input, Operator Grammar Validation, LEADING/TRAILING computation, and Operator Precedence Table construction. Review 2 reuses these modules and adds a fifth stage for string parsing.



Figure 5.1 Existing Grammar Input Stage

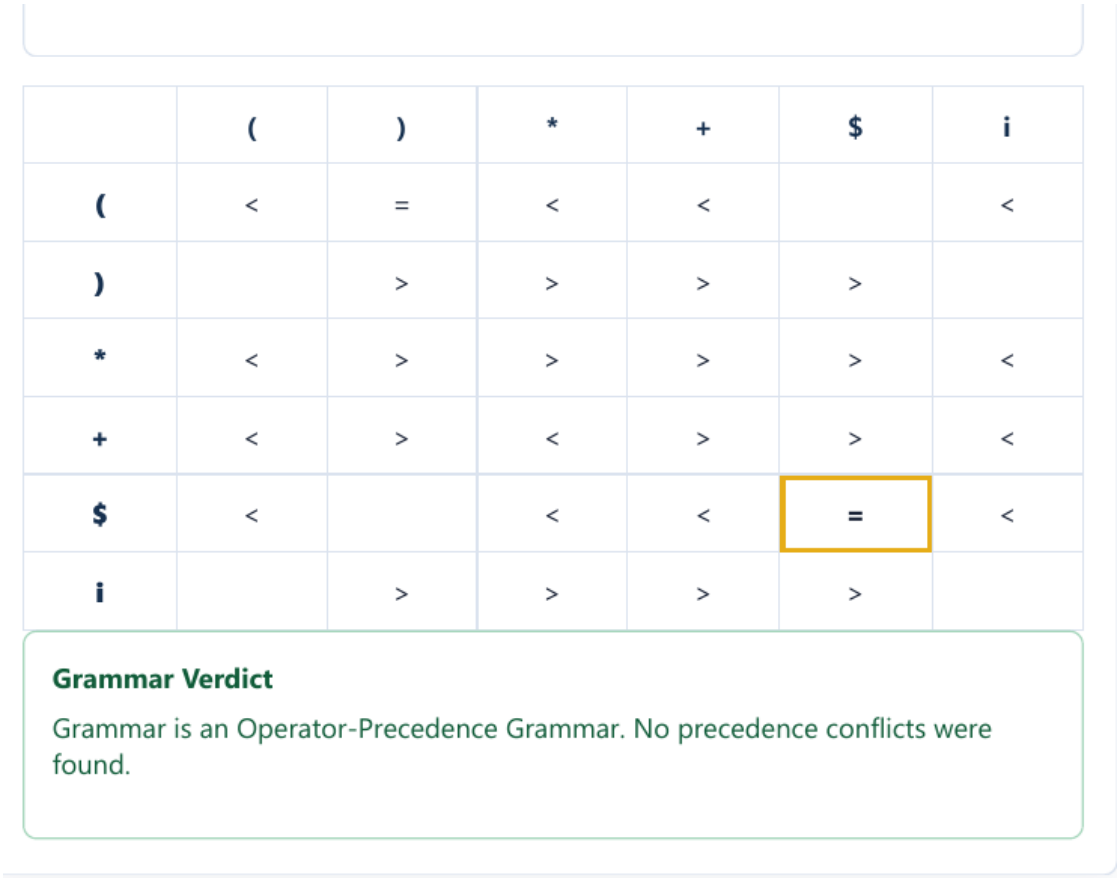


Figure 5.2 Existing Precedence Table Output

5.1.2 Review-2 Parser Stage

The Review-2 parser stage accepts an input string only after the grammar has been validated and the precedence table has been generated without conflicts. The parser maintains a typed stack, tokenized input, current input position, step list, and visited-state set. The UI exposes Previous, Next Step, Run All, and Reset Parsing controls. Each REDUCE step highlights the detected handle and names the grammar production used.

The implementation supports compact notation such as E -> E+T | T and token-separated notation such as Expr -> Expr '+' Term | Term. Longest-match tokenization prevents terminals such as id from being split into i and d.

5.1.3 Error Handling

- Invalid grammar: parser stage remains disabled.
- Precedence conflict: parser stage remains disabled because the action is non-deterministic.
- Unknown terminal: input tokenization stops and reports the symbol and position.

- Missing table relation: parsing terminates at the exact stack/input terminal pair.
- No matching handle: the candidate handle is shown and parsing terminates.
- Ambiguous reduction: all candidate productions are reported and no arbitrary reduction is applied.
- Non-progressing cycle: repeated parser state or maximum-step overflow terminates parsing safely.

5.2 TESTING

Table 5.1 Review-2 Test Cases

Test Case	Grammar / Input	Expected Result	Purpose
TC1	E -> E+T | T; T -> T*F | F; F -> (E) | i; input i+i*i	ACCEPT	Standard expression parsing
TC2	S -> aAb | b; A -> c; input acb	ACCEPT	Different grammar; tests aAb and a = b relation
TC3	Expr -> Expr '+' Term | Term; Term -> Term '*' Factor | Factor; Factor -> '(' Expr ')' | 'id'; input id+id*id	ACCEPT	Multi-character symbols
TC4	S -> AB; A -> a; B -> b	REJECT BEFORE PARSING	Adjacent non-terminals
TC5	S -> aA | epsilon; A -> b	REJECT BEFORE PARSING	Epsilon validation
TC6	Conflict-producing grammar	PARSING DISABLED	Table conflict
TC7	Expression grammar; input i&i	REJECT	Unknown input terminal

CHAPTER 6
RESULTS AND DISCUSSION

6.1 DETAILED RESULTS

6.1.1 Generic Grammar Validation Example

A useful non-arithmetic grammar for verifying grammar independence is S -> aAb | b and A -> c. This grammar is a valid operator grammar and yields the important relation a = b because the production contains terminal-nonterminal-terminal in the form a A b. For input acb, the expected parser trace is shown below.

Table 6.1 Expected Trace for S -> aAb | b, A -> c, Input acb

Step	Stack	Remaining Input	Relation	Action / Production
1	$	acb$	$ < a	SHIFT a
2	$ a	cb$	a < c	SHIFT c
3	$ a c	b$	c > b	REDUCE c -> A
4	$ a A	b$	a = b	SHIFT b
5	$ a A b	$	b > $	REDUCE aAb -> S
6	$ S	$	-	ACCEPT

6.1.2 Invalid Grammar Handling

The previously verified grammar S -> AB, A -> a, B -> b is rejected because A and B are adjacent non-terminals. The application correctly prevents LEADING/TRAILING, table construction, and string parsing from continuing after this structural validation failure.

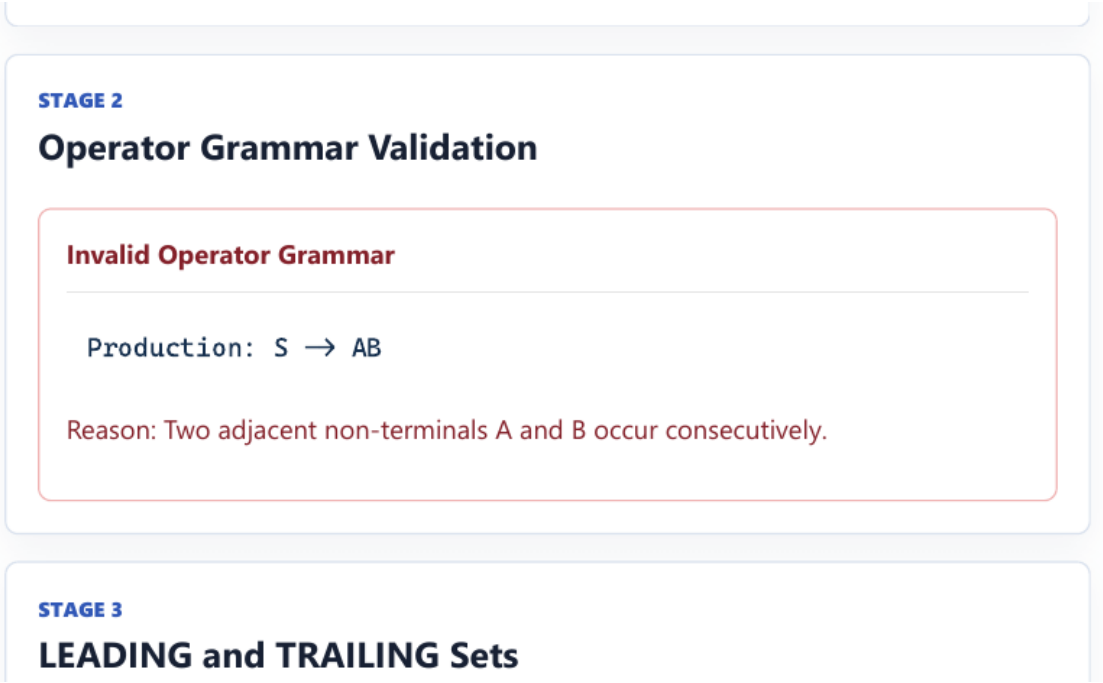
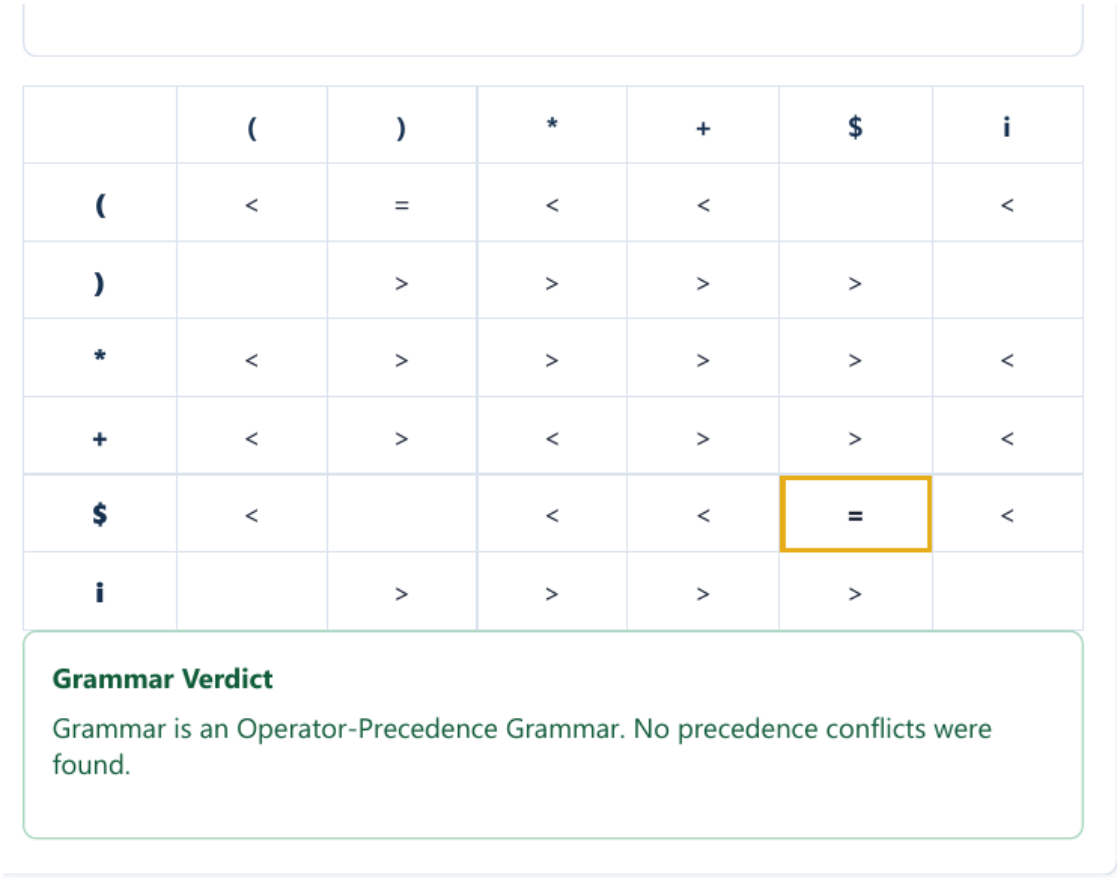


Figure 6.1 Invalid Grammar Rejected for Adjacent Non-Terminals

6.1.3 Precedence Table Foundation

The standard arithmetic grammar from Review 1 produces a valid, conflict-free operator-precedence table. This table becomes the direct control structure used by the Review-2 string parser.



	(	)	*	+	$	i
(	<	=	<	<		<
)		>	>	>	>	
*	<	>	>	>	>	<
+	<	>	<	>	>	<
$	<		<	<	=	<
i		>	>	>	>	

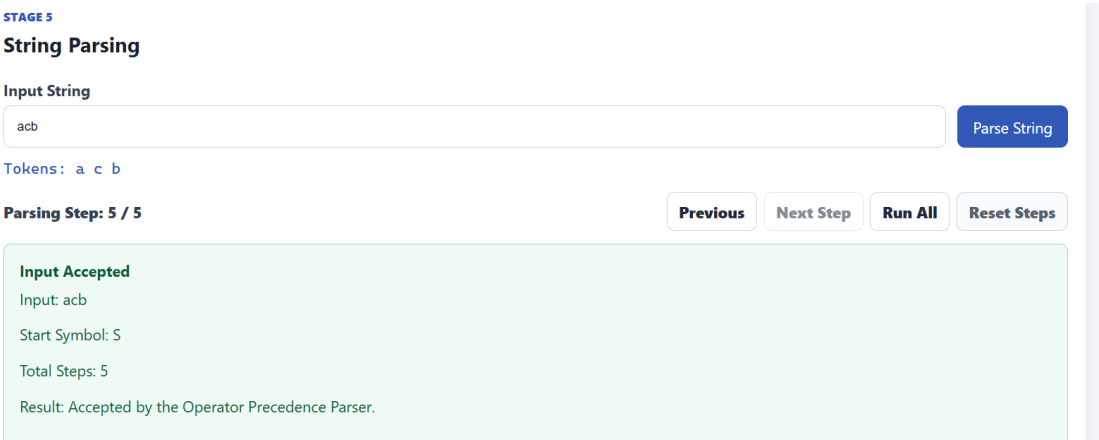
Grammar Verdict

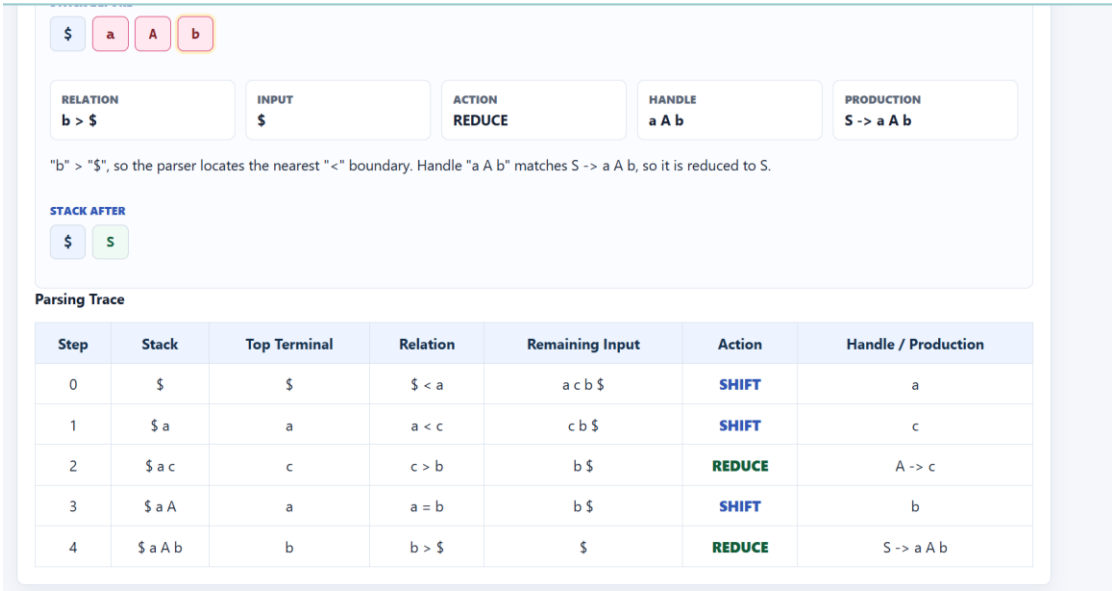
Grammar is an Operator-Precedence Grammar. No precedence conflicts were found.

Figure 6.2 Conflict-Free Precedence Table for Standard Expression Grammar

6.2 OUTPUT SCREENSHOTS

The report currently contains the verified Review-1 foundation screenshots and the exact Review-2 test/trace specification. After running the final Review-2 parser implementation, replace the placeholder below with the actual String Parsing stage showing stack, relation, remaining input, action, handle/production, and ACCEPT/REJECT result. This avoids presenting simulated output as an implemented result.





Parsing Trace

Step	Stack	Top Terminal	Relation	Remaining Input	Action	Handle / Production
0	$	$	$ < a	a c b $	SHIFT	a
1	$ a	a	a < c	c b $	SHIFT	c
2	$ a c	c	c > b	b $	REDUCE	A -> c
3	$ a A	a	a = b	b $	SHIFT	b
4	$ a A b	b	b > $	$	REDUCE	S -> a A b

6.3 PERFORMANCE ANALYSIS AND EVALUATION

The grammar and input sizes expected in a Compiler Design laboratory are small, and all processing is performed interactively in the browser. LEADING/TRAILING computations converge because terminal sets are finite. Precedence-table construction is bounded by the number of production symbol patterns and terminals. During parsing, each SHIFT consumes an input token, while REDUCE changes the stack. The state-cycle detector and maximum-step limit guarantee termination even when unit productions could otherwise produce non-progressing reductions. The primary Review-2 evaluation criteria are correctness, generality across valid operator-precedence grammars, explainability of every parser action, and safe handling of invalid inputs.

CONCLUSION

Review 2 extends the project from grammar analysis to a complete operator-precedence parsing design. The updated system introduces typed grammar tokens, longest-match input tokenization, topmost-terminal comparison, SHIFT/REDUCE execution, correct handle-boundary discovery, production-aware reductions, unit-production closure, parsing traces, and cycle protection. The design is intentionally grammar-independent within the valid operator-precedence grammar class and does not claim to parse arbitrary context-free grammars. The existing Review-1 validation and precedence-table modules remain the foundation for the parser. The next review will complete the second branch of Project 8 by implementing the direct method for Regular Expression to DFA conversion and integrating both modules into the final application.

REFERENCES

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.

[2] Cooper, K. D., & Torczon, L. (2011). Engineering a Compiler (2nd ed.). Morgan Kaufmann.

[3] Appel, A. W. (2002). Modern Compiler Implementation in Java (2nd ed.). Cambridge University Press.